Matthew Owen

Dr. Krishna Kambhampaty

CMPSC 132

April 28, 2026

Final Report for Guessing Game

For my project I chose the number guessing game. This project required me to create a single player, terminal based number guessing game, where you select a number between 1 and 100. Some requirements that this game had were that the game would give you feedback on whether the number was too high, too low, or correct, as well to keep track of attempts. I also needed to use the random.randint(1, 100), a loop to continue until the correct guess, and validate user input. There were also some optional enhancements that I decided to include, which were difficulty levels where a harder difficulty would give you less attempts and an easier one would give you more attempts and a way to see if the player would like to play again.

The first thing I did when writing the code for this project was import random so that way I could actually use the random.randint(1, 100). Next, I started by working on the different functions that would be called in my main game function. For these functions I knew I needed to have one for difficulty selection and then one that makes sure that the guess is a valid integer number between 1 and 100. For my get_difficulty function I set up a couple print statements that would tell you what the game is and what each difficulty is. Then I created an input so the user can give the game their choice of difficulty. To make sure that their choice is one of the selections I used a while loop, that checks to see if the choice is not in “1”, “2”, “3”. This allows the user to make another attempt if they don’t type in one of the choices. Once their selection is one of the choices, I used an if, elif, and else statement, since there are only three choices, to return how many attempts they get based on the difficulty level they chose.

For the get_valid_guess function, I set up some initial conditions such that valid is set to false initially and your guess is set to none. Again, I used a while loop, so the user can keep entering until their guess is valid. The while loop checks the inverse of valid, so once valid changes to true, the while loop stops and returns your guess. To make sure the guess is valid I used a try and except statement, where the try takes an input and turns it into an integer, and then uses an if/else statement to see if its between 1 and 100. The except statement is there if you get a ValueError, i.e. if the guess portion can’t turn your guess into an integer.

Now that these helper functions are done, I was able to move on the game portion of this project. I called that function play_game. Here I set some initial conditions like using the get_difficulty function to get the max attempts that the user will have, then I used the random.randint(1, 100) to actually get the number the user will guess, attempts that were made were set to zero since the user initially hasn't made any, and final a constant that lets the function know if the user has guessed correctly. Using a print statement I then let the user know how many attempts that must find the number. After that I used a while loop again so that way the user can keep guessing a number until their guess is correct or they have run out of attempts. During each attempt the game will tell the user using a print statement how many attempts they have left to guess. The guess is collected using the get_valid_guess function and for each attempt made, the attempts variable is increased by 1. Using an if, elif, else statement, the game will check to see if the answer is too high, too low, or correct and give you feedback based on that. If the guess equals the number the guessed_correctly constant will change to true, if not then a print statement will let you know you lost and what the number is.

The last thing I decided to do was to include a way to play again, I put this under the main function where, using a while loop, as long as play_again is true it will call the play_game function. This is done via another input, it asks the user to enter either y for yes or n for no, if the answer is in "y" then play_again stays true, otherwise its false. If the user says no I thank them for playing the game.

As far as challenges, I didn't really face any. The only thing that was challenging or difficult was coming up with how I wanted it to be structured as in brainstorming the ideas for how I wanted it to work. If anything, the part of coding that I somewhat struggled with was the get_valid_guess function. Although it's one of the shorter functions in my code, I struggled with coming up with a way to make sure the number is an integer, between 1 and 100, and have a way to tell you if it's anything that doesn't work. I was able to work this out by using a try/except statement.

Improvements that could be made could be more difficulties and more edge case handling in certain areas. For difficulties, there could be a larger range or less attempts. For edge case handling, there could be certain ones that I am missing that could be found with more testing.